

Credit Card Fraud Detection Using Machine Learning

Victoria Ma, Ziqi Meng, Jiayi Bai

May 10, 2025

Abstract

This report presents a complete machine learning workflow to detect fraudulent bank account applications using a realistic synthetic dataset. We describe each phase of the pipeline, including dataset background, preprocessing, exploratory data analysis, feature engineering, model evaluation, and lessons learned. The ultimate goal is to minimize false positives while ensuring high fraud detection rates for practical deployment.

1 Introduction

Detecting fraudulent bank account applications is a high-stakes classification task. We base our work on a synthetic dataset derived from the BAF dataset suite released with a NeurIPS 2022 paper. This dataset was generated using CTGAN trained on anonymized real-world data from a large consumer bank. Since the original dataset is confidential, we downloaded the surrogate version from Kaggle provided by the authors. Each instance represents a single bank account application with 32 attributes.

2 Data Understanding and Cleaning

2.1 Data Overview

The dataset consists of 1 million samples with 32 features, including numerical, categorical, binary, and demographic variables. The target is `fraud_bool` (1 = fraud, 0 = not fraud). Fraud rate is approximately 1.1%, indicating severe class imbalance. This imbalance is a central challenge in fraud detection and guides the design of our modeling pipeline.

2.2 Missing Values and Feature Dropping

We first inspected the dataset for missing values, marked by `-1`. Two features (`prev_address_months_count`, `intended_balcon_amount`) had a large proportion of missing entries and were dropped. These features offered limited signal and retaining them could distort model learning. Four other features had moderate missingness and were filled using `NaN` to preserve row-level information without introducing false values. For `bank_months_count`, since most values clustered around 1, we imputed missing entries with 1 to maintain the right-skewed distribution.

3 Exploratory Data Analysis (EDA)

3.1 Distribution Analysis

We visualized distributions using KDE, box plots, and bar charts to identify feature importance and outliers. Features like `name_email_similarity`, `credit_risk_score`, and `current_address_months_count` showed clear separation between fraud and non-fraud, making them strong predictors. Other features like `days_since_request` and `session_length_in_minutes` showed little to no signal and were considered for removal.

3.2 Demographic Insights

We explored feature interactions by demographic slices:

- **Credit Risk:** Fraud cases had significantly higher credit risk scores, which matched expectations and validated this feature’s strength.
- **Age:** Older applicants had higher fraud rates, especially those between 80–90 years old.
- **Income:** Higher-income individuals surprisingly had a higher fraud rate, possibly due to fraudsters targeting higher value accounts.

These patterns informed our decision to retain and prioritize these features.

4 Feature Engineering and Data Preparation

4.1 Correlation Analysis

A heatmap of Pearson correlation coefficients was used to detect multicollinearity among numerical variables. Velocity-based features (`velocity_6h`, `velocity_24h`, `velocity_4w`) were naturally correlated. However, unexpected high correlation between `credit_risk_score` and `proposed_credit_limit` was concerning for QDA, which assumes low inter-feature correlation. This justified our decision to evaluate both linear and non-linear models.

4.2 Preprocessing Pipeline

Step 1: Feature Definition: X contains all columns except `fraud_bool`, which is used as target y .

Step 2: Train-Test Split: We split the data using an 80/20 ratio with `train_test_split()`, using `random_state=42` and `shuffle=True` to ensure reproducibility and randomness. This standard practice avoids leakage and ensures generalization.

Step 3: Feature Transformation:

- **Numeric Features:** Scaled to $[0, 1]$ using `MinMaxScaler` for uniformity across models.
- **Categorical Features:** Encoded using `OneHotEncoder` with `handle_unknown='ignore'` to prevent errors from unseen categories.
- **Combined Pipeline:** A `ColumnTransformer` unified both processes, ensuring consistent pre-processing.

Step 4: Transformation Execution: After fitting, the transformed data was converted to a `DataFrame`, preserving feature names for interpretation.

5 Modeling and Evaluation

5.1 Logistic Regression

We started with logistic regression due to its interpretability and simplicity. The model performed reasonably well:

- $AUC = 0.8711$, $TPR = 0.7755$, $FPR = 0.1956$
- Fraud precision = 4.24%

Despite high AUC, the FPR was close to 20%, which is too high for a real-world fraud detection system. We fixed TPR at 0.8 to ensure we catch most frauds, but this revealed the model’s inability to suppress false positives — a fatal flaw in practical applications.

5.2 Quadratic Discriminant Analysis (QDA)

We tested QDA next, hypothesizing that its ability to model class-specific covariances might better fit complex fraud patterns. However, due to strong multicollinearity (e.g., credit and velocity features), QDA performed poorly:

- $AUC = 0.6304$, fraud precision = 1.23%

This reinforced the need to transition toward tree-based models, which handle feature interactions and skewed distributions better.

5.3 Addressing Class Imbalance with NearMiss

Before applying tree-based models, we needed to address the dataset's severe imbalance (1.1% fraud). NearMiss is an undersampling technique that selects non-fraud examples closest to fraud cases based on feature-space distance. The rationale is:

- Focus on hard-to-classify non-fraud samples near fraud boundaries
- Avoid overwhelming the classifier with easily distinguishable majority examples

We implemented NearMiss to generate a 50:50 balanced dataset. Initially, we were concerned about overfitting due to lost majority-class diversity. However, with 1M+ examples, our professor advised that 50:50 would not significantly degrade generalization. This allowed us to proceed confidently with rebalanced modeling.

5.4 Tree-Based Models

We compared three models (Decision Tree, Random Forest, XGBoost) under both original (90:10) and rebalanced (50:50) settings:

Decision Tree:

- 90:10 $\rightarrow AUC = 0.76$, $TPR = 0.56$, $FPR = 4.49\%$
- 50:50 $\rightarrow AUC = 0.75$, $TPR = 0.66$, $FPR = 12.8\%$

Decision trees adapted well to the full data but became overly optimistic after balancing, leading to higher FPR.

Random Forest:

- 90:10 $\rightarrow AUC = 0.94$, $TPR = 0.52$, $FPR = 0.66\%$
- 50:50 $\rightarrow AUC = 0.91$, $TPR = 0.67$, $FPR = 3.3\%$

Random forests offered excellent robustness. Balancing improved recall while modestly increasing FPR.

XGBoost:

- 90:10 $\rightarrow AUC = 0.958$, $TPR = 0.64$, $FPR = 1\%$
- 50:50 $\rightarrow AUC = 0.953$, $TPR = 0.79$, $FPR = 4.9\%$

XGBoost maintained strong performance across both settings. It achieved the best trade-off, especially when tuned for $TPR = 0.8$.

6 Conclusion and Discussion

After evaluating various modeling strategies, we confirmed that:

- Logistic regression and QDA are insufficient due to the complexity and imbalance in the data
- Class balancing (via NearMiss) combined with tree-based models improves fraud recall while controlling false positives

- XGBoost is our best candidate for deployment, especially when TPR needs to be guaranteed

Important Feature Insights: One of the key outcomes of this project was identifying which features consistently contribute the most to fraud detection across different models. We conducted feature importance analysis using Decision Tree, Random Forest, and XGBoost. The top-ranked features varied slightly between models but showed strong consensus:

- `velocity_24h` and `zip_count_4w` were ranked in the top 2 across all three models.
- `name_email_similarity`, `credit_risk_score`, and `current_address_months_count` consistently ranked in the top 10.
- Features like `session_length_in_minutes` and `days_since_request` performed poorly in KDE and box plot analysis, and were confirmed to be of low importance (Rank 9/10) — leading us to drop them.

This cross-model analysis strengthens our understanding of fraud patterns and helps streamline future modeling by focusing on a core set of impactful features.

Feature	Decision Tree	Random Forest	XGBoost
<code>zip_count_4w</code>	(Rank 1)	(Rank 1)	(Rank 4)
<code>velocity_24h</code>	(Rank 2)	(Rank 2)	(Rank 1)
<code>housing_status</code>	(Rank 3)	(Rank 6)	
<code>bank_branch_count_8w</code>	(Rank 4)	(Rank 3)	(Rank 10)
<code>proposed_credit_limit</code>	(Rank 5)	(Rank 7)	
<code>velocity_6h</code>	(Rank 6)	(Rank 5)	(Rank 3)
<code>velocity_4w</code>	(Rank 7)	(Rank 4)	(Rank 2)
<code>name_email_similarity</code>	(Rank 8)	(Rank 9)	(Rank 5)
<code>credit_risk_score</code>	(Rank 9)	(Rank 8)	(Rank 7)
<code>current_address_months_count</code>	(Rank 10)	(Rank 10)	(Rank 6)
<code>session_length_in_minutes</code>			(Rank 8)
<code>days_since_request</code>			(Rank 9)

Figure 1: Top 10 important features by model (Decision Tree, Random Forest, XGBoost)

Future Directions:

- Apply SMOTE or ADASYN for oversampling instead of undersampling
- Experiment with ensemble techniques (e.g., stacking)
- Explore unsupervised methods such as One-Class SVM
- Use cost-sensitive learning to better optimize precision-recall trade-offs
- Build human-in-the-loop fraud review system

7 References

- S. Jesus et al., *Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation*, NeurIPS 2022.
- BAF Dataset on Kaggle: <https://www.kaggle.com/datasets/>